

**Passez votre système IA du POC au MVP et
réalisez votre portfolio de Data Engineer**

RAG

RAG
LLM / IA

Le RAG : révolutionner l'IA avec la
génération augmentée de récupération



Passez votre système IA du POC au MVP et réalisez votre portfolio de Data Engineer

RAG

SOMMAIRE

- ☐ Contexte du projet (mission)
- ☐ Présentation de la mission
- ☐ Description de la démarche technique complète
- ☐ Conclusion

RAG
LLM / IA

Le RAG : révolutionner l'IA avec la
génération augmentée de récupération



Présentation de la mission

Contexte du projet

- Puls Events est une plateforme permettant de consulter des événements culturels en temps réel selon des critères comme le lieu, la date ou la thématique.
- Les données proviennent principalement de sources externes (notamment OpenAgenda) et sont hétérogènes, non normalisées et non vectorisées.
- La plateforme ne dispose pas encore d'un assistant intelligent capable de comprendre des questions en langage naturel ou de proposer des recommandations personnalisées.
- L'entreprise souhaite tester l'intégration d'un chatbot IA basé sur un pipeline RAG afin d'améliorer la recherche d'événements et l'expérience utilisateur.
- Le projet consiste à transformer un POC existant en un MVP cloud-ready, scalable et intégrable dans Puls Events.
- Le MVP inclut un backend RAG complet et un frontend léger (Streamlit) pour démonstration et validation.
- Les résultats permettront de décider d'un passage à l'échelle et d'une future industrialisation



Présentation de la mission

Enjeux du projet

- Améliorer l'expérience utilisateur grâce à des recommandations culturelles personnalisées
- Rendre les données OpenAgenda exploitables via une interface conversationnelle.
- Garantir la qualité des recommandations (fraîcheur, cohérence géographique, pertinence).
- Simplifier l'accès à l'information culturelle pour les utilisateurs.
- Mesurer la performance du système avec un jeu de tests annotés.
- Fournir une base technique solide pour les équipes produit et marketing avant industrialisation.



Présentation de la mission

Objectif global du projet

- Transformer le POC existant en un MVP moderne, scalable et prêt pour la production.
- Remplacer les composants du POC par une architecture cloud robuste, performante et basée sur des outils récents.
- Industrialiser l'ingestion, la vectorisation, la recherche sémantique et le pipeline RAG pour un usage réel.
- Fournir un MVP stable, testable et démontrable via un frontend léger (Streamlit).
- Préparer une base technique solide permettant un passage à l'échelle et une intégration complète dans Puls Events.



Présentation de la mission

Description

- **Mission Etape 1 : Plan du projet**
- **Mission Etape 2: Macro backlog**
- **Mission Etape 3: Architecture technique détaillé**
- **Mission Etape 4: Gestion du projet**
- **Mission Etape 5: Estimation des coûts**
- **Mission Etape 6: Bilan**



Mission Etape 1: Plan du projet



RAG

RAG
LLM / IA

Le RAG : révolutionner l'IA avec la
génération augmentée de récupération

Mission Etape 1: Plan du projet

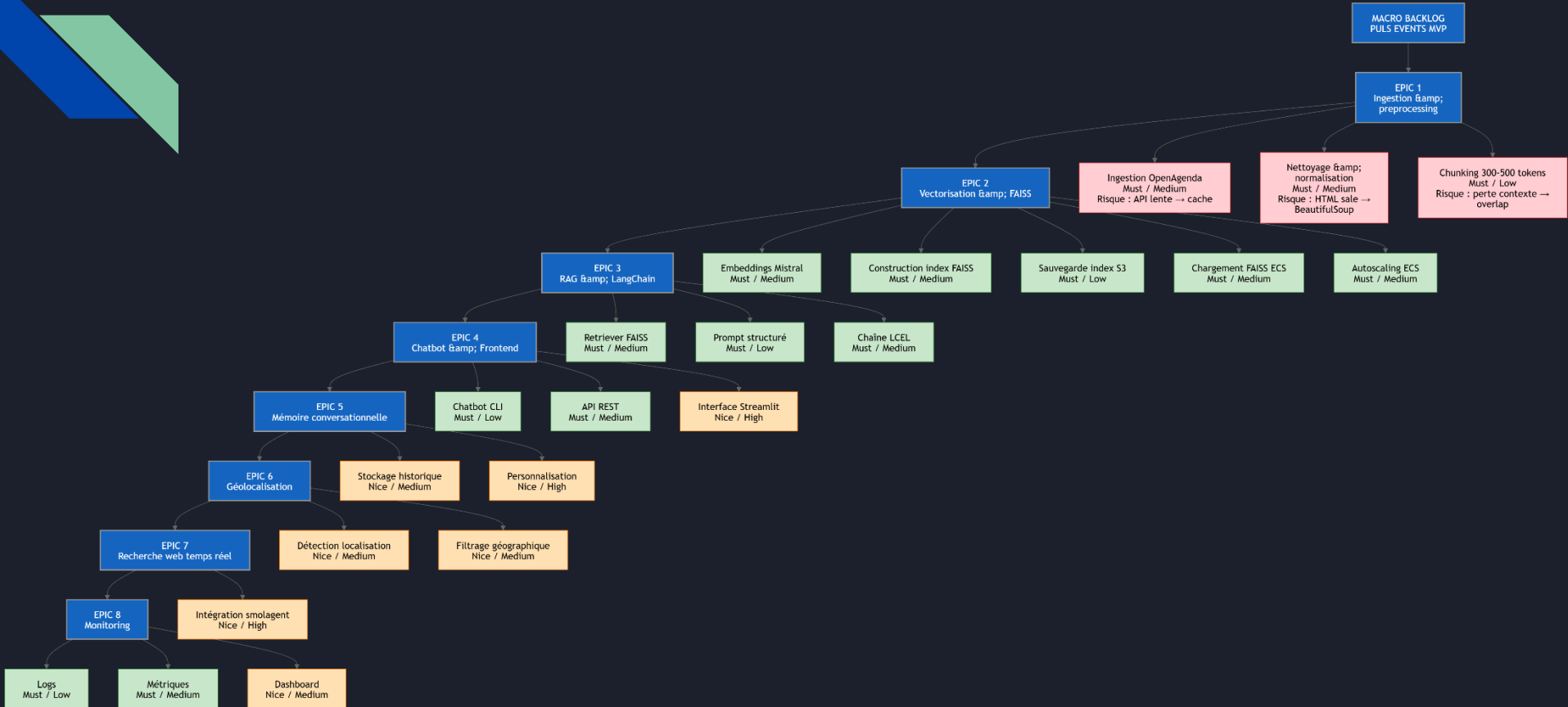
Jalons	Description	Date
M0	Validation du design	S1
M1	Ingestion et preprocessing	S2
M2	Embeddings et FAISS	S3
M3	RAG et chatbot backend	S4
M4	Mémoire et géolocalisation	S5
M5	Monitoring	S6
M6	Frontend Streamlit	S7
M7	Déploiement MVP	S8

Mission Etape 2: Macro backlog

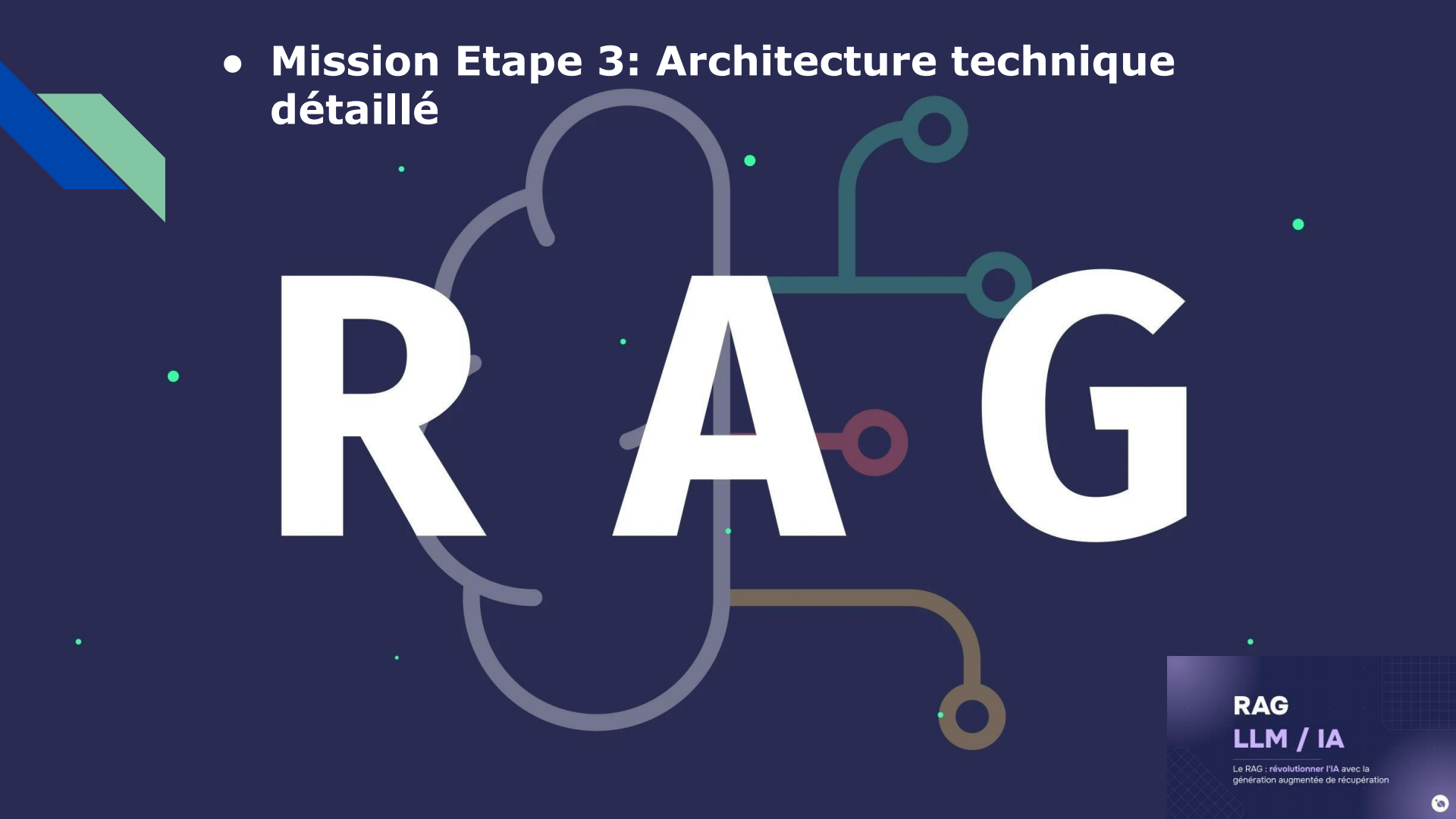
RAG

RAG
LLM / IA

Le RAG : révolutionner l'IA avec la
génération augmentée de récupération



- **Mission Etape 3: Architecture technique détaillé**



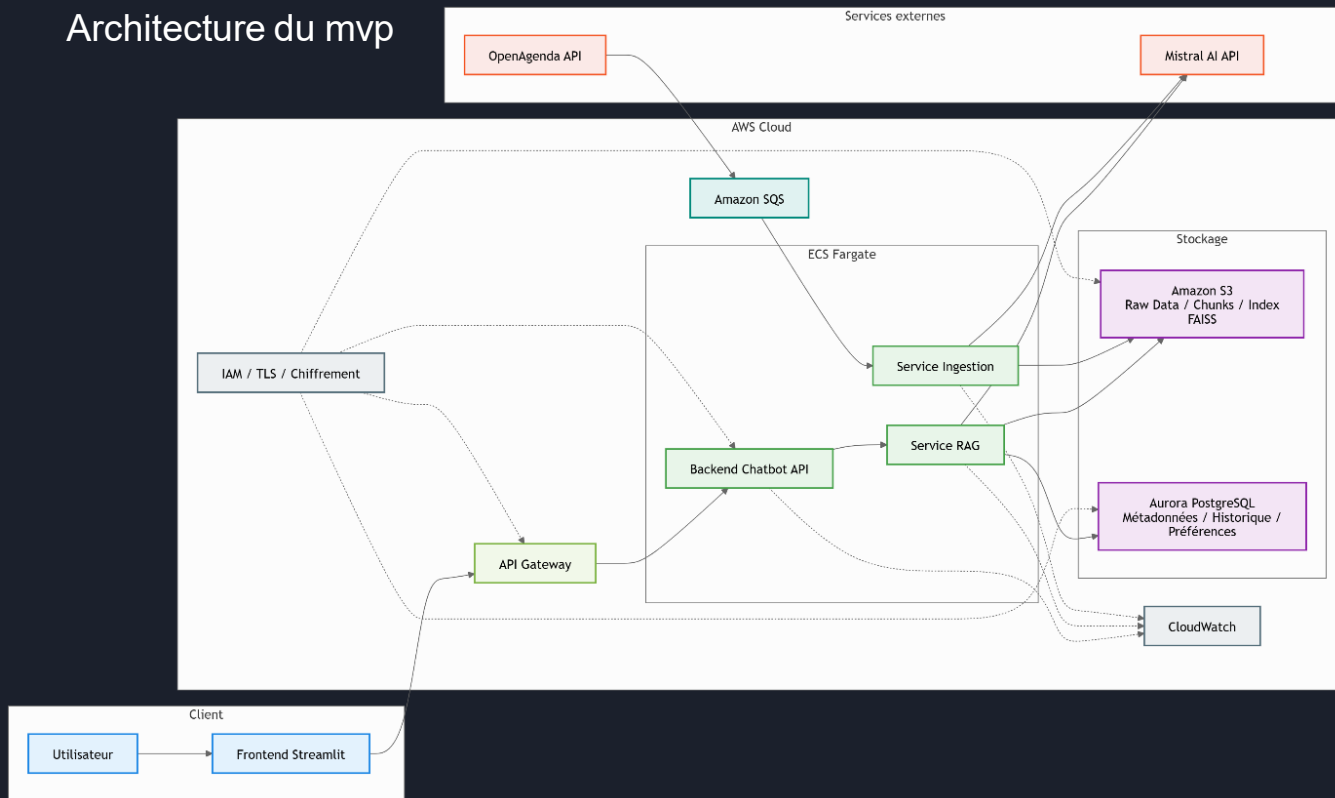
RAG

RAG
LLM / IA

Le RAG : révolutionner l'IA avec la
génération augmentée de récupération

Mission Etape 3: Architecture technique détaillé

Architecture du mvp



Mission Etape 3: Architecture technique détaillé

1. S3 — Stockage brut, chunks, index FAISS

Option	Avantages	Inconvénients	Justification
S3	Très scalable, économique, haute disponibilité	Pas de requêtes SQL	Choisi pour stocker données brutes, chunks et index FAISS
EFS	Partage de fichiers, POSIX	Coût plus élevé	Non nécessaire pour un MVP
RDS	SQL, structuré	Pas adapté aux fichiers volumineux	Non pertinent pour stockage brut

Choix retenu : S3 → Le plus économique, scalable et adapté au stockage d'index FAISS

Mission Etape 3: Architecture technique détaillé

2. SQS — File d'ingestion scalable

Option	Avantages	Inconvénients	Justification
SQS	Simple, scalable, fiable	Pas de traitement natif	Choisi pour absorber les pics d'ingestion
Kafka	Très scalable, streaming	Complexité élevée	Surdimensionné pour un MVP
EventBridge	Intégration AWS	Moins adapté au batch	SQS reste plus simple

Choix retenu : SQS → Suffisant, scalable, faible coût, parfait pour ingestion asynchrone.

Mission Etape 3: Architecture technique détaillé

3. ECS Fargate ingestion workers

Option	Avantages	Inconvénients	Justification
ECS Fargate	Serverless, autoscaling, isolation	Coût > Lambda	Choisi pour ingestion + embeddings
Lambda	Simple, économique	Limité en CPU/RAM	Insuffisant pour embeddings
EC2	Flexible	Maintenance élevée	Non adapté pour un MVP moderne

Choix retenu : ECS Fargate → Permet ingestion + embeddings + nettoyage dans un environnement scalable.

Mission Etape 3: Architecture technique détaillé

4. ECS Fargate RAG service

Option	Avantages	Inconvénients	Justification
ECS Fargate	Autoscaling, stateless, performant	Coût supérieur	Choisi pour charger FAISS en RAM
Lambda	Serverless	Temps de cold start, RAM limitée	Incompatible avec FAISS
EC2	Contrôle total	Maintenance lourde	Non retenu

Choix retenu : ECS Fargate → Nécessaire pour charger FAISS en RAM et scaler horizontalement..

Mission Etape 3: Architecture technique détaillé

5. API Gateway — Endpoint chatbot

Option	Avantages	Inconvénients	Justification
API Gateway	Serverless, sécurisé, quotas	Coût par appel	Choisi pour exposer l'API du chatbot
ALB	Moins cher	Moins de fonctionnalités	Moins adapté pour API publique
Nginx sur EC2	Flexible	Maintenance	Non retenu

Choix retenu : API Gateway → Standard AWS pour exposer une API serverless.

Mission Etape 3: Architecture technique détaillé

6. Frontend Streamlit — Interface MVP

Option	Avantages	Inconvénients	Justification
Streamlit	Rapide, simple, idéal MVP	Pas optimisé production	Choisi pour démonstration MVP
React	Moderne, scalable	Temps de dev plus long	Surdimensionné pour MVP
Vue.js	Léger	Plus complexe	Non retenu

Choix retenu : Streamlit → Permet un MVP rapide, simple et démontrable.

Mission Etape 3: Architecture technique détaillé

7. Mistral API — Génération LLM

Option	Avantages	Inconvénients	Justification
Mistral API	Français natif, rapide, économique	Dépendance externe	Choisi pour cohérence embeddings + génération
OpenAI	Très performant	Coût plus élevé	Non retenu
Llama local	Pas de coût	Besoin GPU	Impossible dans MVP cloud

Choix retenu : Mistral API → Cohérence embeddings + génération, coût maîtrisé..

Mission Etape 3: Architecture technique détaillé

8. CloudWatch — Logs et métriques

Option	Avantages	Inconvénients	Justification
CloudWatch	Intégré AWS, simple	Coût selon volume	Choisi pour logs + métriques
Grafana	Très visuel	Setup plus complexe	Option secondaire
Datadog	Très complet	Coût élevé	Non retenu pour MVP

Choix retenu : CloudWatch → Suffisant pour monitoring MVP.

Mission Etape 3: Architecture technique détaillé

9. IAM — Sécurité

Option	Avantages	Inconvénients	Justification
IAM	Permissions minimales, natif AWS	Complexité des policies	Choisi pour sécuriser tous les composants
Vault	Très sécurisé	Setup lourd	Surdimensionné
Secrets Manager	Stockage secrets	Coût	Utilisé en complément

Choix retenu : IAM → Standard AWS pour sécurité fine-grainée.

Mission Etape 3: Architecture technique détaillé

10. Aurora PostgreSQL Serverless v2 — Métadonnées, préférences, historique

Option	Avantages	Inconvénients	Justification
Aurora Serverless v2	Scalable, SQL, performant	Coût supérieur	Choisi pour stockage structuré
RDS PostgreSQL	Stable	Pas autoscaling	Moins flexible
DynamoDB	Très scalable	Pas SQL	Non adapté aux relations

Choix retenu : Aurora Serverless v2 → Base SQL scalable, parfaite pour historique conversationnel.

Mission Etape 3: Architecture technique détaillé

11. pgvector — Option évolutive pour remplacer FAISS

Option	Avantages	Inconvénients	Justification
pgvector	Managé, SQL + vector search	Moins rapide que FAISS	Option future si FAISS devient trop lourd
FAISS	Ultra rapide	Non managé	Utilisé pour MVP
Pinecone	Managé	Coût élevé	Non retenu

Choix retenu : FAISS pour MVP, pgvector en évolution → FAISS = performance maximale ; pgvector = scalabilité future.

Mission Etape 3: Architecture technique détaillé

12. Justification du provider

Critère	AWS	GCP	Azure
NLP	Très bon	Bon	Moyen
Serverless	Excellent	Excellent	Bon
Monitoring	Excellent	Excellent	Bon
Coût	Bon	Moyen	Bon
Écosystème	Excellent	Bon	Moyen

- **Mission Etape 4: Gestion du projet**

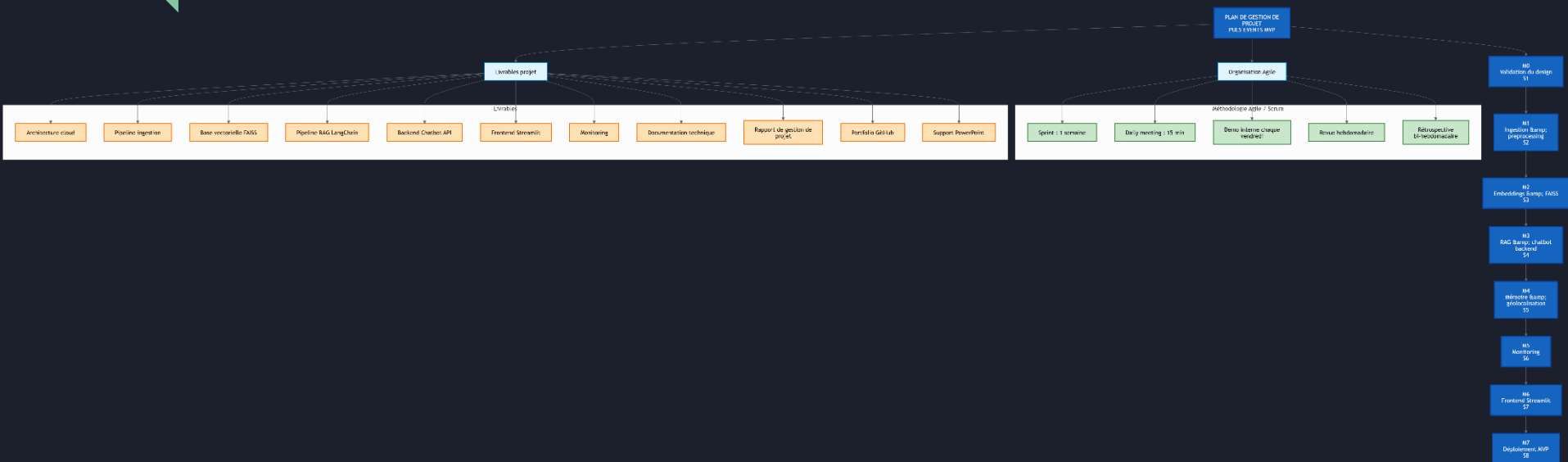
RAG

RAG
LLM / IA

Le RAG : révolutionner l'IA avec la
génération augmentée de récupération

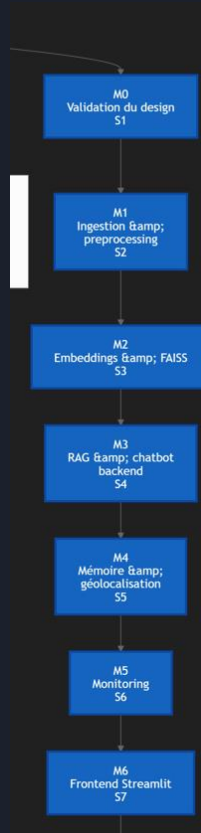
Mission Etape 4: Gestion du projet

Lecture globale du schéma



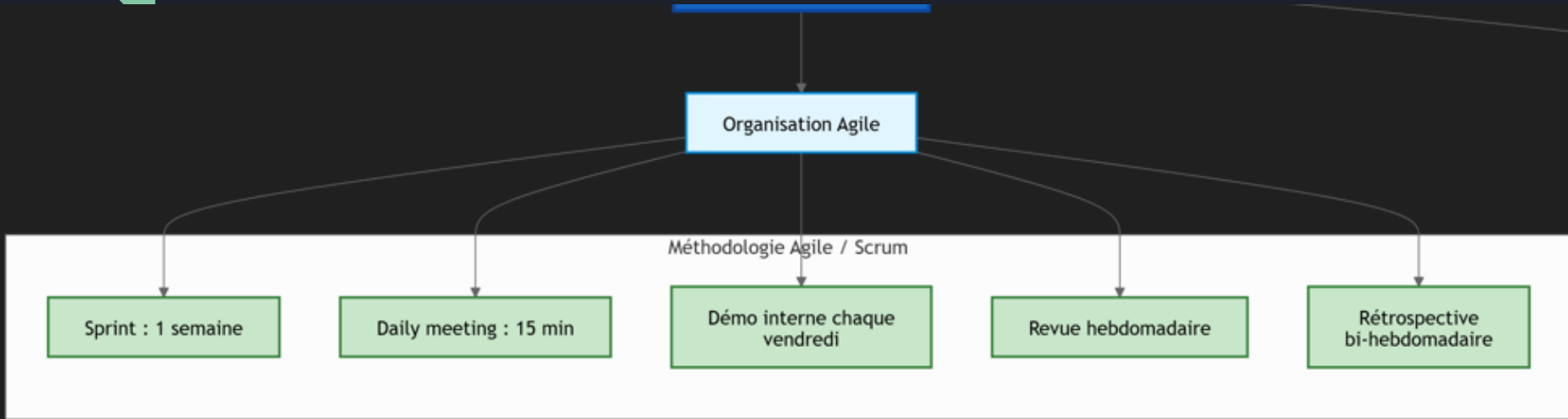
Mission Etape 4: Gestion du projet

1. Roadmap projet



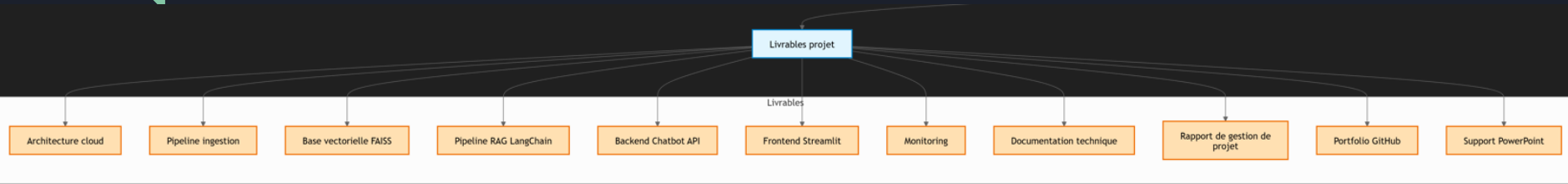
Mission Etape 4: Gestion du projet

2. Organisation Agile / Scrum



Mission Etape 4: Gestion du projet

3. Livrables projet



- **Mission Etape 5: Estimation des coûts**

RAG

RAG
LLM / IA

Le RAG : révolutionner l'IA avec la
génération augmentée de récupération

Mission Etape 5: Estimation des coûts

1-Build

Élément	Durée	Coût estimé
Ingestion	1 semaine	2 500 €
RAG	2 semaines	5 000 €
Déploiement cloud	1 semaine	2 500 €
Frontend Streamlit	1 semaine	2 500 €
Tests et optimisation	1 semaine	2 500 €

Total BUILD : 15 000 €

2-OPEX

LOW (100 utilisateurs/jour) : 103 €/mois MEDIUM (1 000 utilisateurs/jour) : 350 €/mois HIGH (10 000 utilisateurs/jour) : 1 360 €/mois

- **Mission Etape 6: Bilan**

RAG

RAG
LLM / IA

Le RAG : révolutionner l'IA avec la
génération augmentée de récupération

Mission Etape 6: Bilan

Élément	Description	Analyse
<u>Du POC au MVP</u>	Passage d'un POC local à un MVP cloud scalable	Le POC utilisait un RAG local, FAISS en local et une interface CLI. Le MVP repose désormais sur une architecture cloud moderne : ingestion scalable, FAISS chargé en RAM via ECS Fargate, API Gateway, monitoring CloudWatch, sécurité IAM et un frontend Streamlit pour la démonstration. - Mistral : cohérence embeddings + génération, coût maîtrisé. - FAISS : performance maximale pour la recherche vectorielle. - LangChain : orchestration simple et modulaire du pipeline RAG. - AWS : meilleure combinaison serverless + scalabilité + monitoring. - Streamlit : rapidité de mise en place pour un MVP démontrable.
<u>Justification des choix techniques</u>	Sélection des outils modernes et scalables	- Nettoyage et normalisation des données hétérogènes. - Définition d'un chunking optimal pour la recherche sémantique. - Gestion du filtrage par année et cohérence temporelle. - Latence variable de l'API Mistral. - Coûts liés aux appels LLM. - Conception d'un frontend simple mais efficace pour la validation utilisateur.
<u>Défis rencontrés</u>	Contraintes techniques et fonctionnelles	

Conclusion

Le projet Puls Events évolue d'un POC technique vers un MVP robuste, scalable et prêt pour la production. L'architecture proposée permet une montée en charge progressive, un coût maîtrisé, et une intégration fluide dans la plateforme existante. Le frontend Streamlit permet une validation rapide de l'expérience utilisateur avant une intégration complète dans la plateforme Puls Events.

Les prochaines étapes incluent :

- ❑ mémoire conversationnelle avancée,
- ❑ géolocalisation enrichie,
- ❑ recherche web temps réel,
- ❑ intégration front-end complète dans Puls Events.

